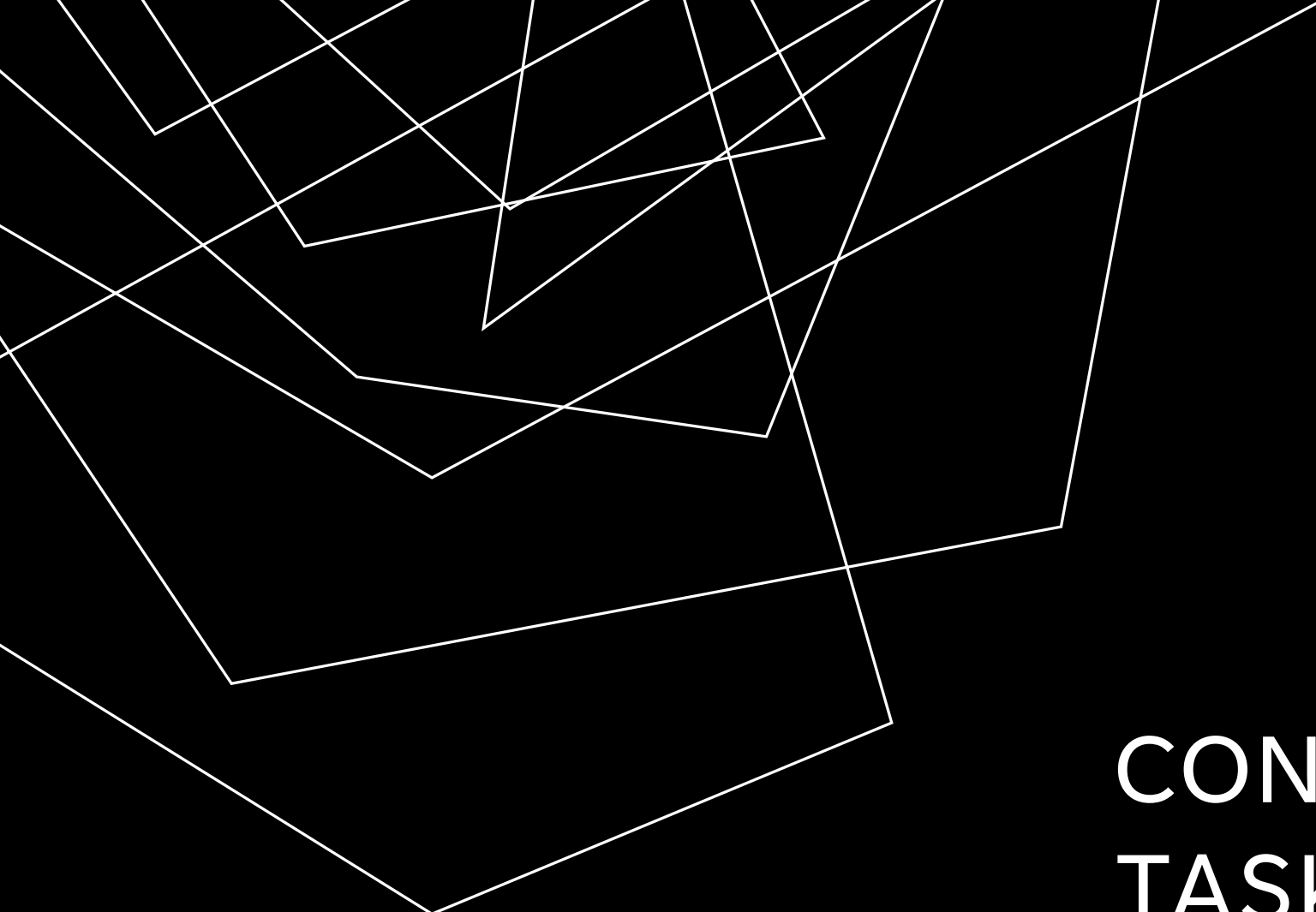
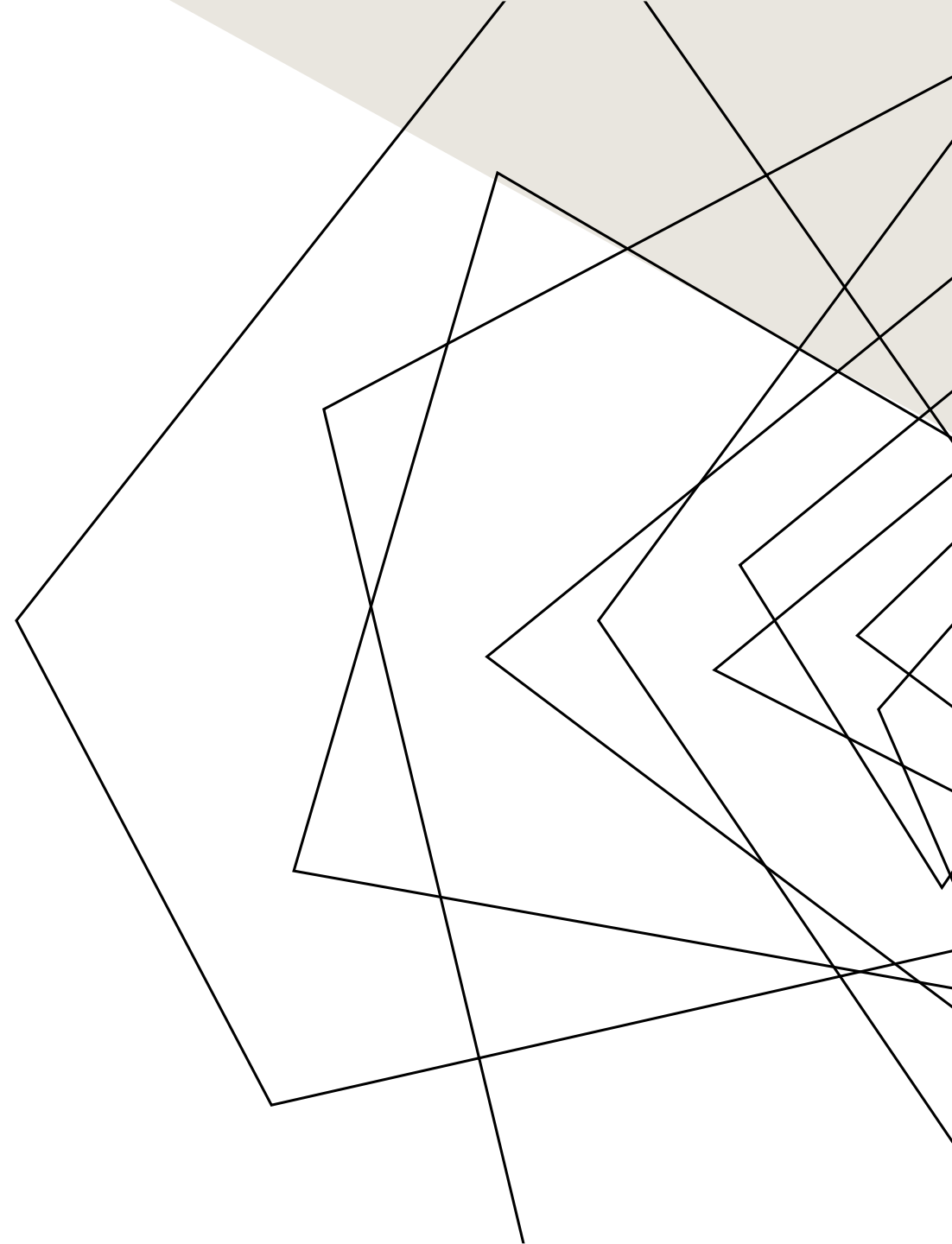


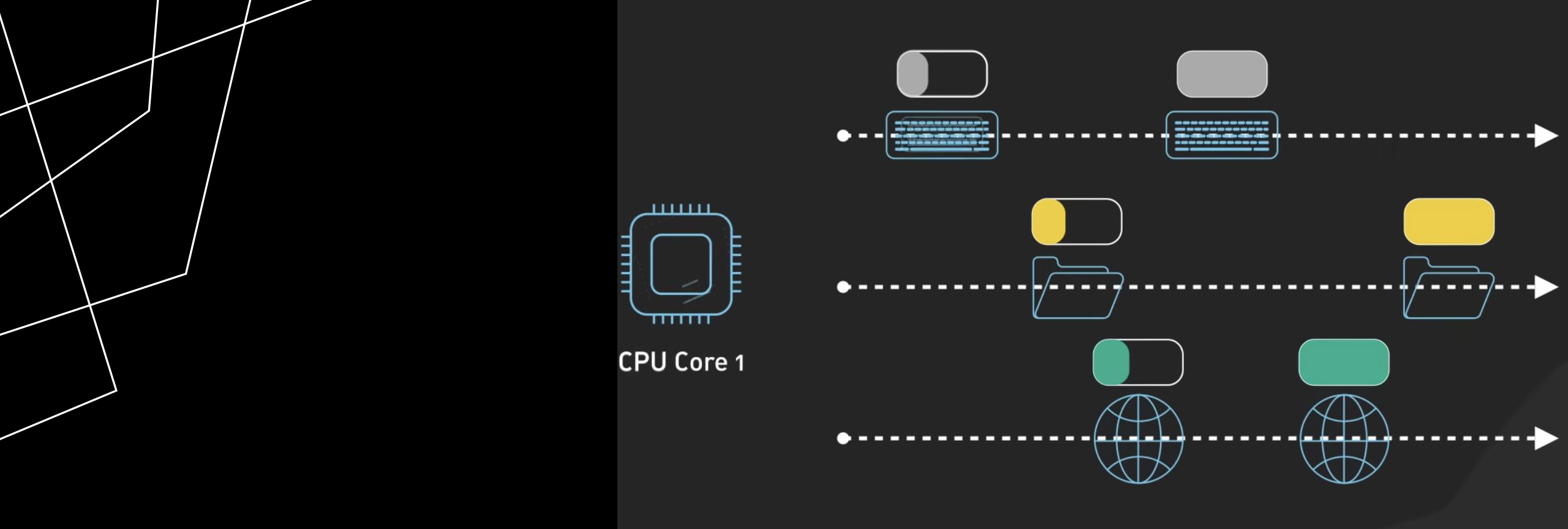


CONCURRENT
TASKS. FUTURES

CONTENTS

- **Concurrent tasks**
- **Futures and Promises**
- **Brief comparisons:**
 - **Concurrent vs Parallel**
 - **Futures vs Promises**





CONCURRENT TASKS

WHAT ARE CONCURRENT TASKS?

- Concurrency = multiple tasks making progress at the same time
- Tasks share the same CPU, switching between them rapidly
- Focus: structure and responsiveness, not simultaneous execution
- Often used for I/O-bound operations

Key idea

Concurrency is about dealing with many things at once, not doing many things at once.

Concurrency



DEALING with multiple tasks at once

Parallelism



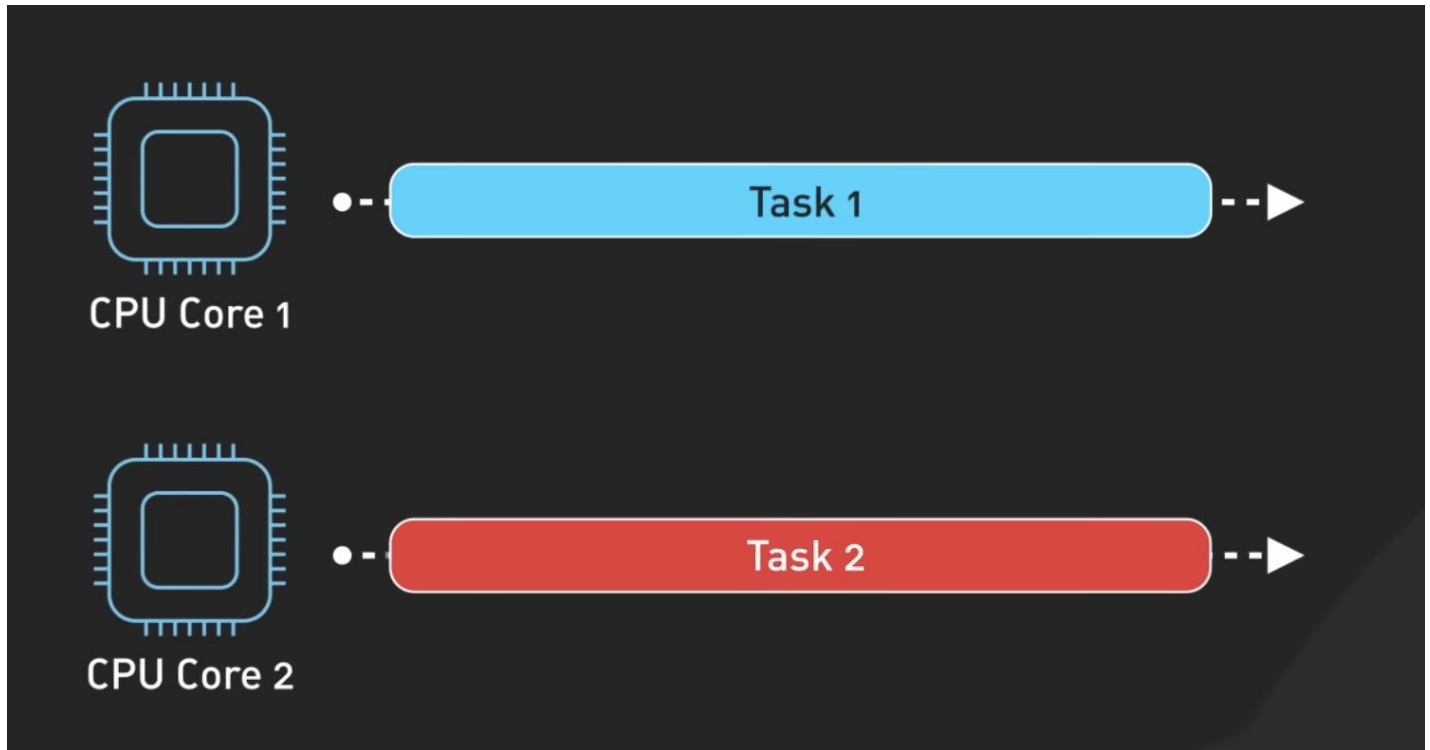
EXECUTING multiple tasks at once

HOW CONCURRENT TASKS WORK?

- Managed via threads or async tasks
- CPU performs context switching between tasks
- Each task maintains its own state
- Overhead: saving/restoring state

WHAT ARE PARALLEL TASKS?

- Parallelism = multiple tasks executing at the same time
- Requires multiple CPU cores
- Focus: performance and speed
- Ideal for CPU-bound computations



Concurrency



DEALING with multiple tasks at once

Parallelism



EXECUTING multiple tasks at once

HOW PARALLEL TASKS WORK?

- Tasks are divided into independent subtasks
- Each core executes one subtask
- Combines results for final output
- Common in data analysis, rendering, ML training

QUICK COMPARISON

Feature	Concurrency	Parallelism
Meaning	Managing multiple tasks	Executing multiple tasks simultaneously
CPU cores	1 or more	2 or more
Goal	Responsiveness	Performance
Ideal for	I/O-bound tasks	CPU-bound tasks
Usage	Web Servers; GUI Responsiveness; I/O-heavy apps	Scientific simulations; Video or Image rendering; Machine Learning

RECAP

Concurrency → handles many tasks efficiently



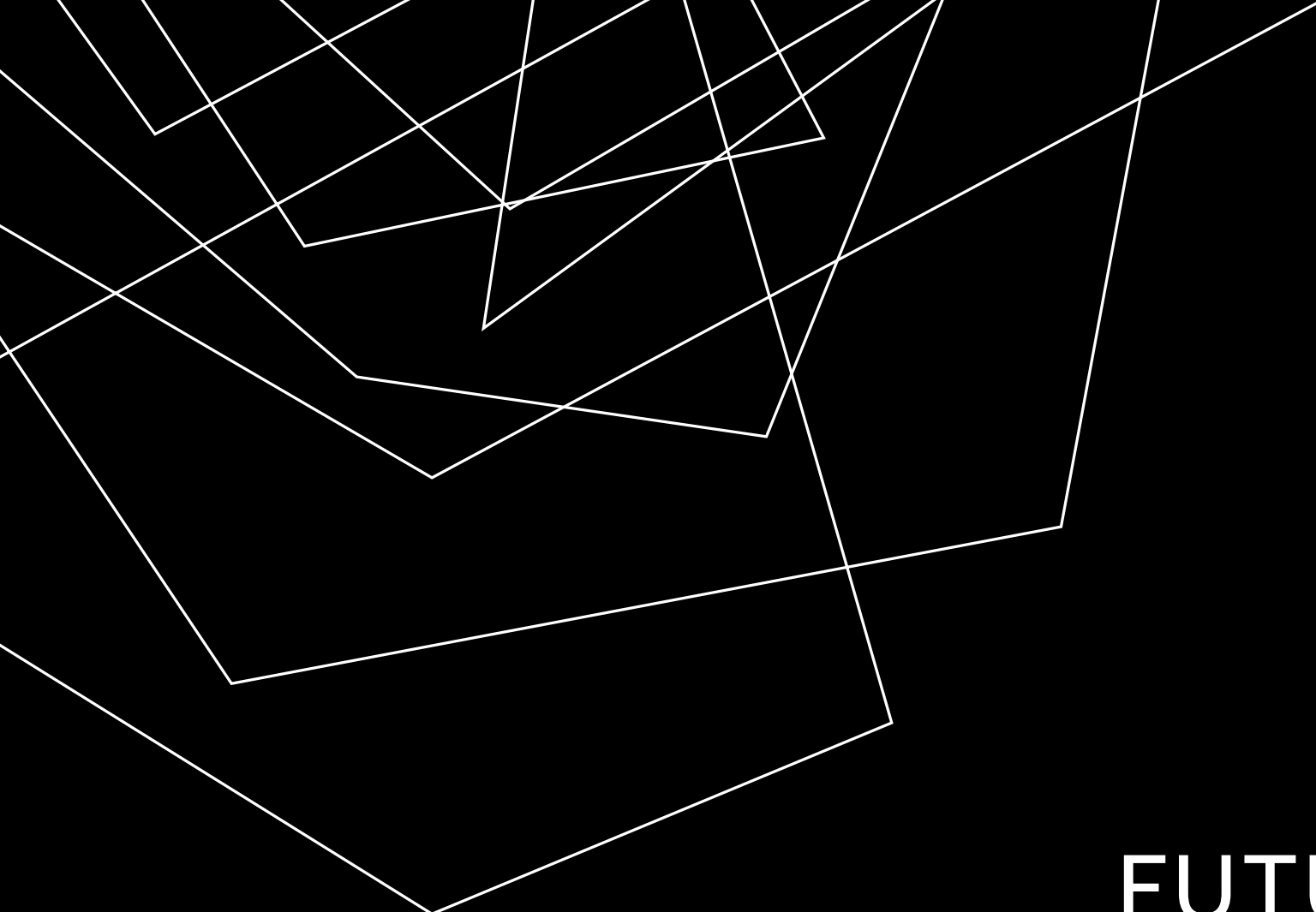
Parallelism → executes many tasks simultaneously



Both improve system performance and scalability



Together, they build responsive and powerful software



FUTURES AND PROMISES

WHY FUTURES AND PROMISES?



Enable **asynchronous** and **non-blocking** computation



Simplify **thread communication**



Avoid **low-level synchronization** tools (like locks or mutexes)



Improve **performance and responsiveness**

TERMINOLOGY:FUTURES

class template object

a read only object containing data

- the data may not be available or computed in the present
- data is promised to be available to you in the future

get() method is the main purpose of a future object

- calling this method will block until the data is available
- returns a value or throws an exception

TERMINOLOGY: PROMISES

class template object

promise object has an associated future object

- which is automatically instantiated when a promise is created
- the only public constructor for a future object creates an empty or invalid future

promise object guarantees it will take "the value returned" by the function and place in the future object at some point in time

HOW DO THEY WORK TOGETHER?

1. A Promise is created.
2. It automatically generates an associated Future.
3. The Promise is passed to a worker thread or async function.
4. The worker sets the result on the Promise.
5. The Future becomes ready, and `get()` retrieves the value.

```
CompletableFuture<Integer> future = new CompletableFuture<>();

new Thread(() -> {
    int result = compute();
    future.complete(result); // fulfills the promise
}).start();

System.out.println(future.get()); // waits for result
```

WHEN TO USE FUTURES AND PROMISES

Use Them When:

Tasks take time but can run independently

You need to **continue other work** while waiting for results

Working with **I/O, network calls, or database queries**

Combining results from multiple concurrent computations

EXAMPLE: PARALLEL COMPUTATION

Run multiple independent computations and combine results.



```
CompletableFuture<Integer> a = CompletableFuture.supplyAsync(() -> computeA());  
CompletableFuture<Integer> b = CompletableFuture.supplyAsync(() -> computeB());  
  
int sum = a.get() + b.get(); // waits for both  
System.out.println(sum);
```


EXCEPTION HANDLING



If the computation fails, the **Future holds an exception**



`get()` throws the exception when called



A **broken promise** occurs if a promise is never fulfilled



In Java, use `completeExceptionally()` to handle failures

BENEFITS

- Improved **responsiveness** and **resource usage**
- Cleaner asynchronous code without manual synchronization
- Built-in **error propagation** and **result handling**
- Encourages **modular** and **composable** design

Concept	Description	Example (Java)
Future	Read-only placeholder for a value	<code>Future<Integer> f = executor.submit(...)</code>
Promise	Write-end used to fulfill the value	<code>CompletableFuture.complete()</code>
Async	High-level abstraction managing both	<code>CompletableFuture.supplyAsync(...)</code>

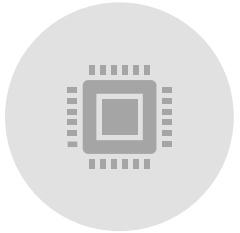
RECAP



Future = placeholder for an upcoming result



Promise = the mechanism that provides the result



Together, they enable asynchronous, safe, and scalable programming



Use them for long-running or parallelizable operations

BRIEF COMPARISON

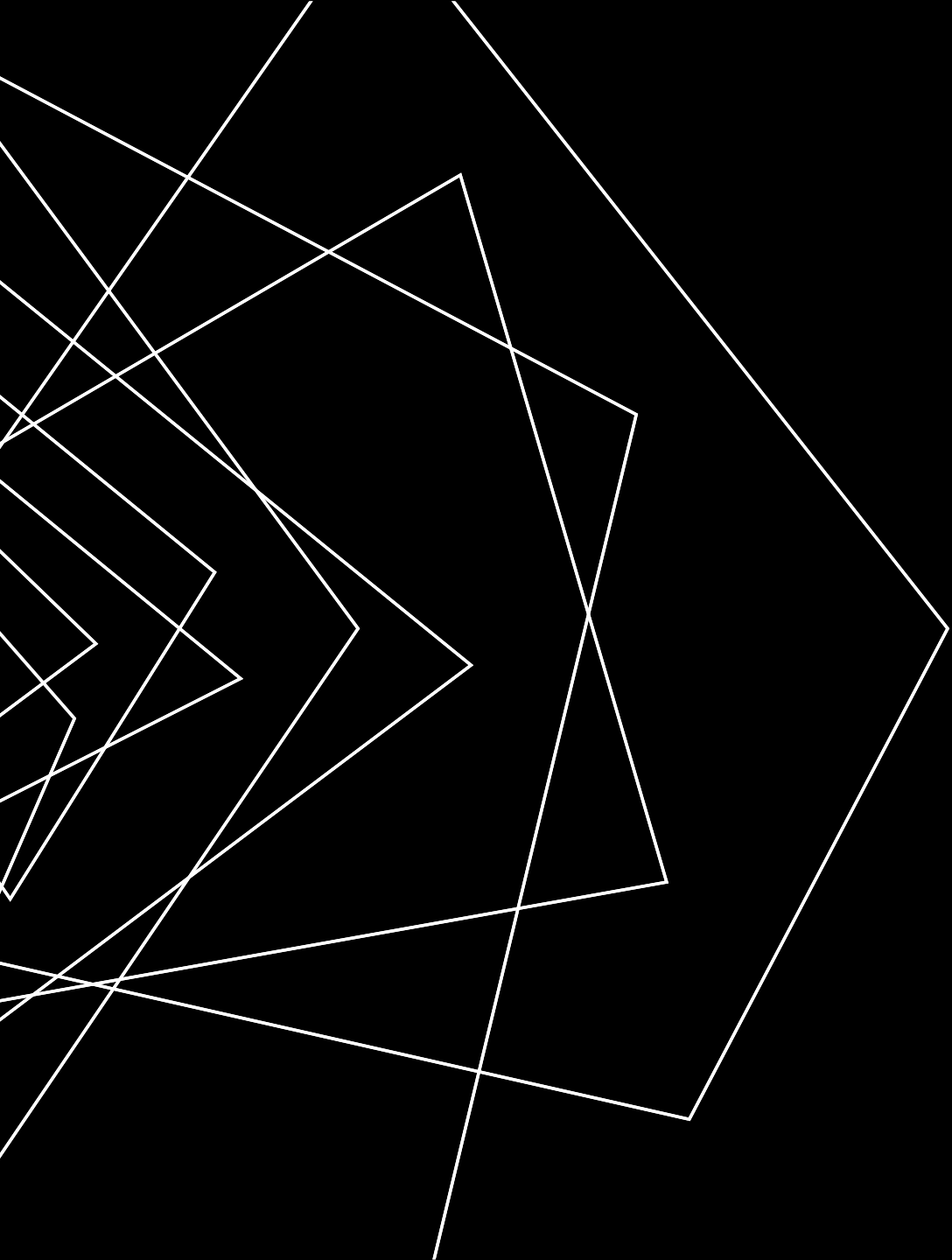
	Description	Analogy
Concurrent Tasks	Multiple tasks progressing during overlapping times	Like multitasking one CPU core
Parallel Tasks	Multiple tasks executed truly at the same time on multiple cores	Like several people working simultaneously
Futures	Placeholder for a result of an asynchronous computation	Like a receipt for an order
Promises	The write-end that fulfills or rejects a future	Like the chef who prepares the meal

SUMMARY

Concurrent Tasks: enable progress of multiple operations without simultaneous execution

Futures: represent results of asynchronous computations

Both improve efficiency, responsiveness, and maintainability in modern systems



THANK YOU